

ЛАБОРАТОРНАЯ РАБОТА 5

Имитационное моделирование

Богданюк Анна Васильевна

Российский университет дружбы народов



ВВОДНАЯ ЧАСТЬ



ЦЕЛЬ РАБОТЫ

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.



ОСНОВНАЯ ЧАСТЬ



БАЗОВЫЕ ЭЛЕМЕНТЫ И ПРИНЦИП РАБОТЫ

В основе сети Петри лежит двудольный ориентированный мультиграф — это означает, что вершины в ней делятся на два типа, дуги всегда соединяют вершины разных типов, и между двумя вершинами может быть несколько дуг.

Позиции (Places) — «Условия»

Пассивные элементы, которые обозначаются кружками. Они описывают состояния системы, наличие ресурса или выполнение некоторого условия.

Метки (Tokens): Внутри позиции могут находиться фишки (маркеры), обозначаемые точками или числом. Они символизируют количество ресурсов или кратность выполнения условия. Состояние сети определяется распределением фишек по всем позициям и называется маркировкой.



Переходы (Transitions) — «События»

Активные элементы, обозначаемые прямоугольниками. Они моделируют действия, события или процессы, которые могут произойти в системе.

Дуги (Arcs) — «Связи»

Направленные связи между позициями и переходами. - Входные дуги (от позиции к переходу) указывают, какие условия и сколько фишек необходимо для срабатывания события. - Выходные дуги (от перехода к позиции) показывают, какие условия станут истинными после события и сколько фишек там появится. Срабатывание перехода есть атомарная операция, которая происходит мгновенно и меняет маркировку сети. Если разрешено несколько переходов, любой из них может сработать.



ФОРМАЛЬНОЕ ОПРЕДЕЛЕНИЕ

Сеть Петри (N-схема) — это четвёрка: $N = (P, T, F, M0)$,

где: - $P = p1, p2, \dots, pn$ — конечное множество позиций (мест); - $T = t1, t2, \dots, tm$ — конечное множество переходов; - $F \subseteq (P \times T) \cup (T \times P)$ — множество дуг (отношения инцидентности); - $M0 : P \rightarrow \mathbb{N}$ — маркировка, задающая начальное распределение фишек



ОБЕДАЮЩИЕ ФИЛОСОФЫ. КЛАССИЧЕСКАЯ ПОСТАНОВКА

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов. Правила поведения философов: - Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест. - Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа. - Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся. - Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.



Ограничения: - Вилками нельзя пользоваться одновременно двум философам. - Философ не может отнять вилку у соседа — только дождаться, пока тот её положит



ПРОБЛЕМА ВЗАИМНОЙ БЛОКИРОВКИ (DEADLOCK)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поест — система замирает. Это и есть взаимная блокировка.



РЕШЕНИЯ ПРОБЛЕМЫ

Арбитр (официант) — вводится дополнительный процесс, который разрешает есть не более чем $N-1$ философам одновременно.



ВЫПОЛНЕНИЕ РАБОТЫ

Создаём рабочее пространство для работы (рис. 1).

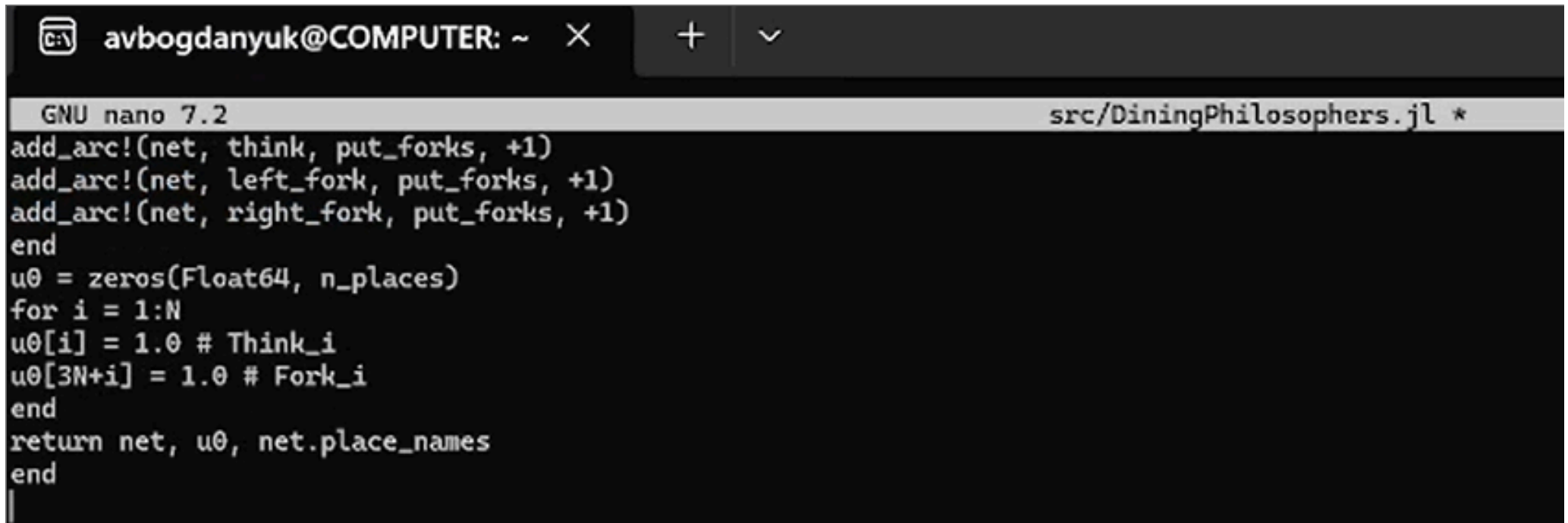
```
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05$ nano setup_project.j  
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05$ julia setup_project.  
|
```

Рисунок 1: Создаём рабочее пространство



ВЫПОЛНЕНИЕ РАБОТЫ

Создаём код модели (рис. 2).



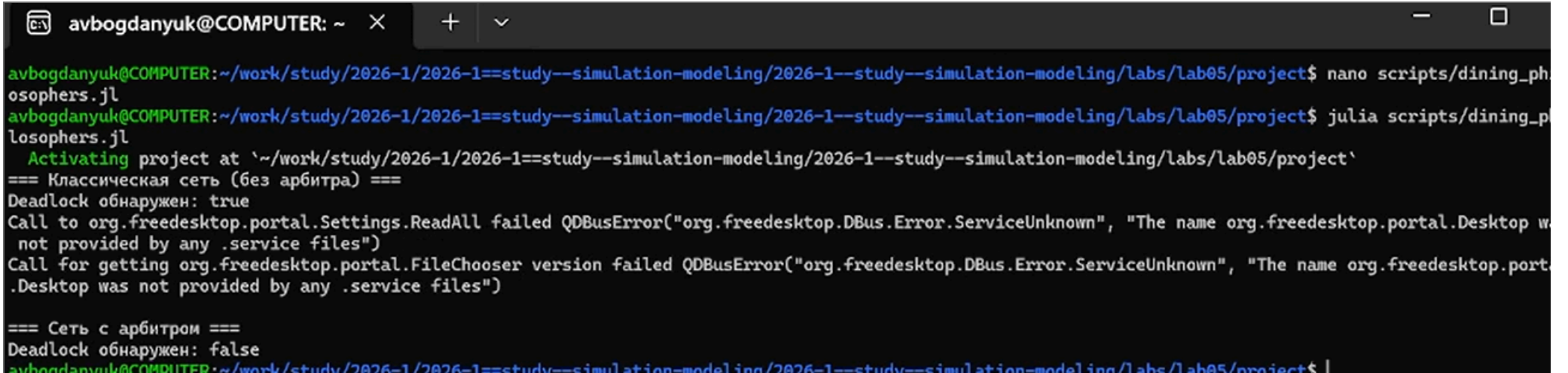
```
avbogdanyuk@COMPUTER: ~ × + v
GNU nano 7.2 src/DiningPhilosophers.jl *
add_arc!(net, think, put_forks, +1)
add_arc!(net, left_fork, put_forks, +1)
add_arc!(net, right_fork, put_forks, +1)
end
u0 = zeros(Float64, n_places)
for i = 1:N
    u0[i] = 1.0 # Think_i
    u0[3N+i] = 1.0 # Fork_i
end
return net, u0, net.place_names
end
|
```

Рисунок 2: Создаём src/DiningPhilosophers.jl



ВЫПОЛНЕНИЕ РАБОТЫ

Создаём файл `scripts/dining_philosophers.jl` (рис. 3).

A terminal window with a dark background and light-colored text. The window title bar shows 'avbogdanyuk@COMPUTER: ~' and standard window controls. The terminal content shows a user running a Julia script. The first command is 'nano scripts/dining_philosophers.jl'. The second command is 'julia scripts/dining_philosophers.jl'. The output shows the activation of a project, followed by a deadlock detection message 'Deadlock обнаружен: true'. Then, there are two error messages related to 'org.freedesktop.portal' services. Finally, the output shows 'Deadlock обнаружен: false' and the prompt returns to the shell.

```
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ nano scripts/dining_philosophers.jl
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ julia scripts/dining_philosophers.jl
  Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project`
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true
Call to org.freedesktop.portal.Settings.ReadAll failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")
Call for getting org.freedesktop.portal.FileChooser version failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")

=== Сеть с арбитрами ===
Deadlock обнаружен: false
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$
```

Рисунок 3: Базовый эксперимент



ВЫПОЛНЕНИЕ РАБОТЫ

Зачем?

Скрипт выполняет основное моделирование и сравнение двух вариантов сети

Петри: - классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock); - модифицированная модель с арбитром, которая должна предотвращать deadlock



ВЫПОЛНЕНИЕ РАБОТЫ

Что делает?

- Создаёт классическую сеть Петри для $N=5$ философов с помощью `build_classical_network`.
- Запускает стохастическую симуляцию (алгоритм Гиллеспи) на время $t_{\max} = 50.0$.
- Сохраняет полную историю маркировок в CSV-файл (`dining_classic.csv`).
- Анализирует последнее состояние на предмет deadlock с помощью `detect_deadlock` и выводит результат в консоль.
- Строит графики эволюции маркировки по всем позициям (Think, Hungry, Eat, Fork) и сохраняет их как `classic_simulation.png`.
- Повторяет шаги 1–5 для сети с арбитром (`build_arbiter_network`), сохраняя результаты в `dining_arbiter.csv` и `arbiter_simulation.png`.



Создаём файлы `scripts/dining_philosophers_animation.jl` и `scripts/dining_philosophers_report.jl`. Один для создания анимации эксперимента, а второй - для создания отчёта по результатам базовой эксперимента (рис. 4).



ВЫПОЛНЕНИЕ РАБОТЫ

```
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ nano scripts/dining_philosophers_animation.jl
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ julia scripts/dining_philosophers_animation.jl
Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project`
Call to org.freedesktop.portal.Settings.ReadAll failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")
Call for getting org.freedesktop.portal.FileChooser version failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")
[ Info: Saved animation to /home/avbogdanyuk/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ nano scripts/dining_philosophers_report.jl
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ julia scripts/dining_philosophers_report.jl
Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project`
```

Рисунок 4: Скрипта для создания анимации и отчёт
Создаём файлы scripts/tangel.jl для того, чтобы преобразовать скрипты в производные форматы и преобразовываем (рис. 5).



ВЫПОЛНЕНИЕ РАБОТЫ



```
avbogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-mod
/tangle.jl scripts/dining_philosophers_animation.jl
анерация из: scripts/dining_philosophers_animation.jl
Info: generating plain script file from '~/work/study/2026-1/2026-1==stud
ots/dining_philosophers_animation.jl'
Info: writing result to '~/work/study/2026-1/2026-1==study--simulatio
sophers_animation/dining_philosophers_animation.jl'
/ Чистый скрипт: /home/avbogdanyuk/work/study/2026-1/2026-1==study--s
g_philosophers_animation/dining_philosophers_animation.jl
Info: generating markdown page from '~/work/study/2026-1/2026-1==stud
dining_philosophers_animation.jl'
Info: writing result to '~/work/study/2026-1/2026-1==study--simulatio
sophers_animation/dining_philosophers_animation.qmd'
/ Quarto: /home/avbogdanyuk/work/study/2026-1/2026-1==study--simulatio
sophers_animation/dining_philosophers_animation.qmd
Info: generating notebook from '~/work/study/2026-1/2026-1==study--s
g_philosophers_animation.jl'
Info: writing result to '~/work/study/2026-1/2026-1==study--simulatio
losophers_animation/dining_philosophers_animation.ipynb'
/ Notebook: /home/avbogdanyuk/work/study/2026-1/2026-1==study--simula
nilosophers_animation/dining_philosophers_animation.ipynb
```



Рисунок 5: Создаём производные форматы
Результат программы `scripts/dining_philosophers.jl` (рис. 6).



ВЫПОЛНЕНИЕ РАБОТЫ



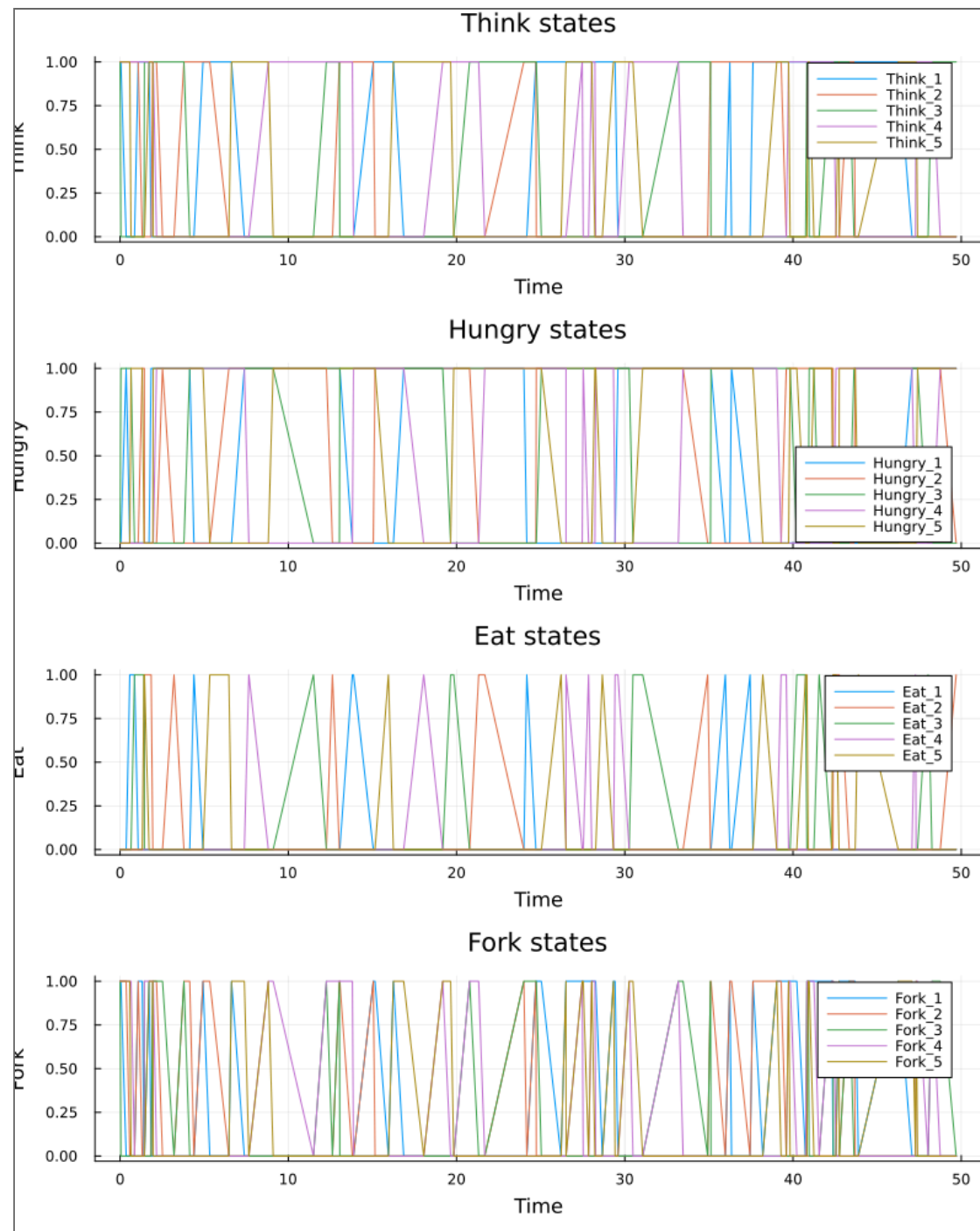


Рисунок 6: Результат scripts/dining_philosophers.jl
Результат программы scripts/dining_philosophers.jl (рис. 7).



ВЫПОЛНЕНИЕ РАБОТЫ



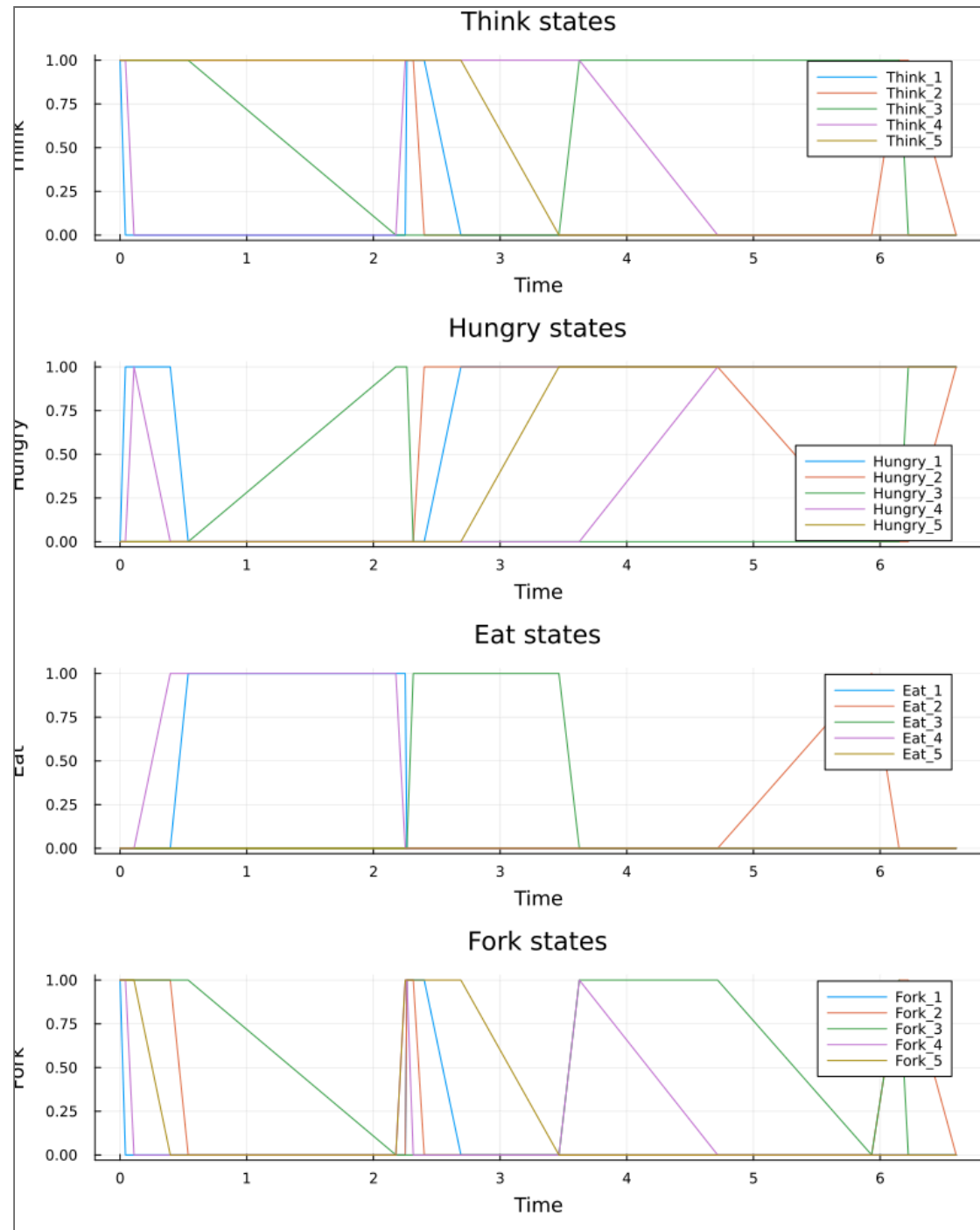


Рисунок 7: Результат scripts/dining_philosophers.jl

Результат программы scripts/dining_philosophers_animation.jl (рис. 8).



ВЫПОЛНЕНИЕ РАБОТЫ



Время = 0.26

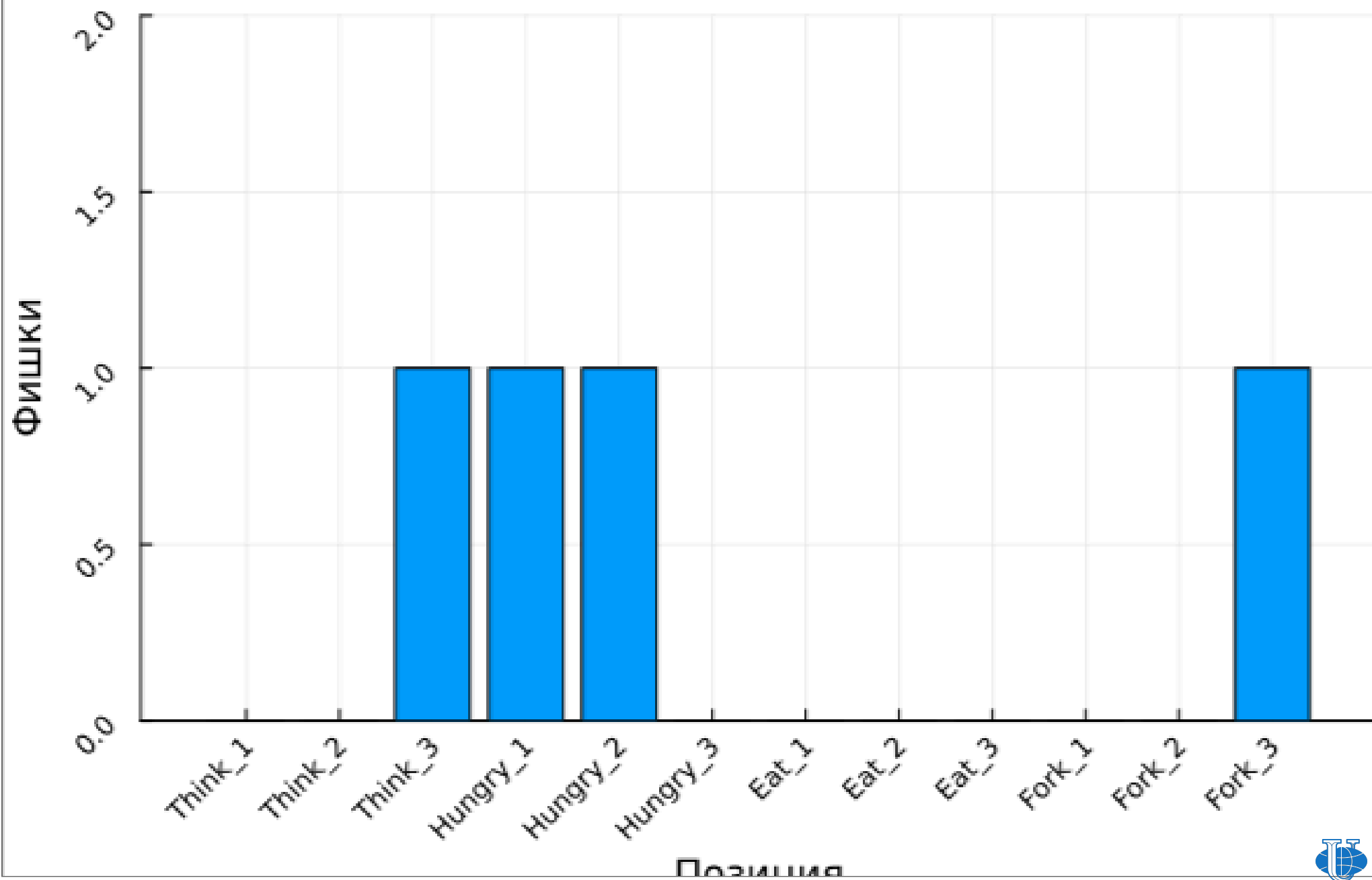


Рисунок 8: Результат `scripts/dining_philosophers_animation.jl`



ВЫПОЛНЕНИЕ РАБОТЫ

Результат программы `scripts/dining_philosophers_report.jl` (рис. 9).

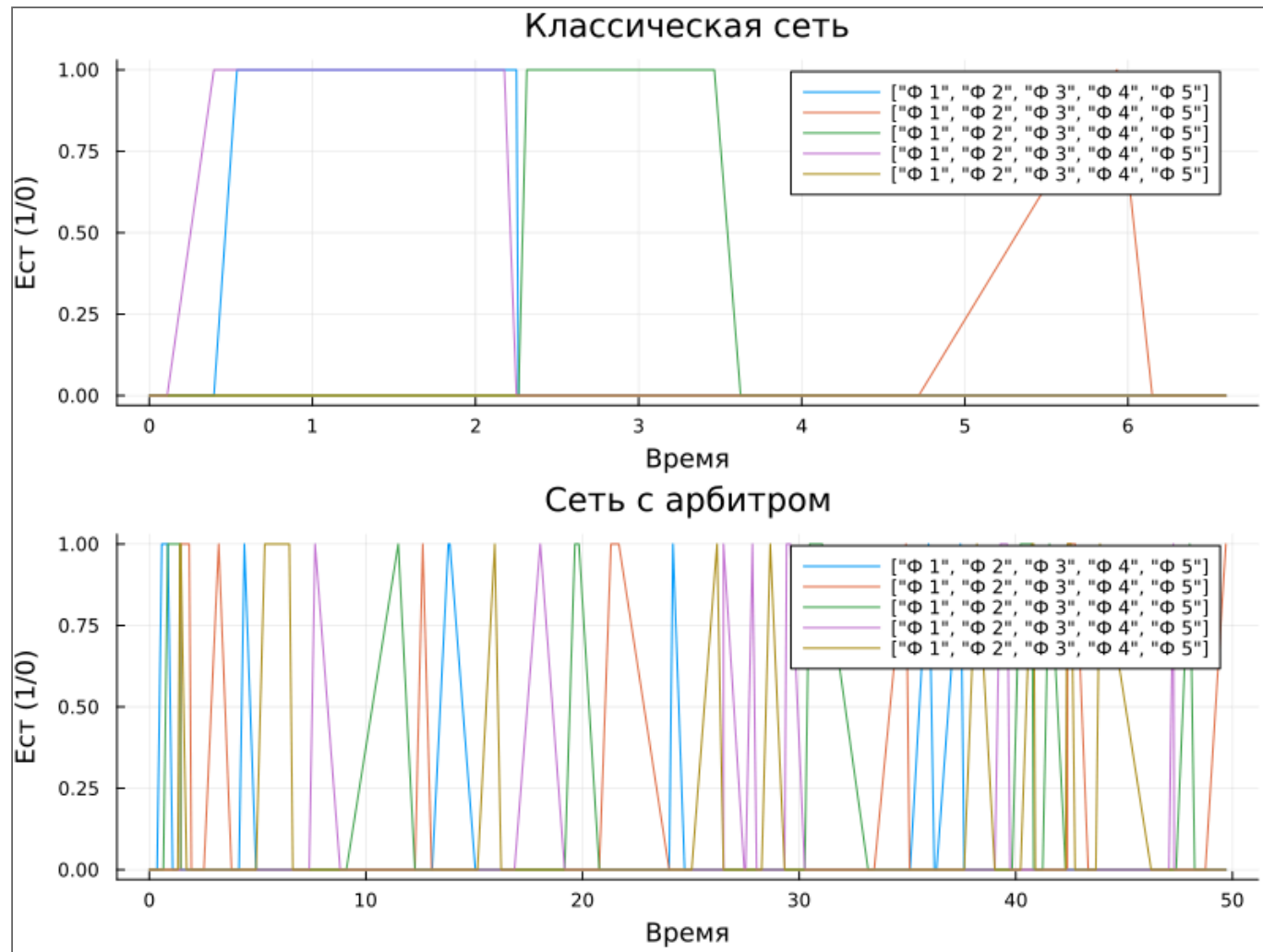


Рисунок 9: Результат scripts/dining_philosophers_report.jl



ВЫВОДЫ

В ходе выполнения лабораторной работы была построена сеть Петри для пяти философов, моделируя захват и освобождение вилок, обнаружена состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую, было проведено имитационное моделирование (стохастическое и детерминированное) и выявлено наличие deadlock, была модифицировать сеть, чтобы предотвратить deadlock, были проанализированы результаты и был оформлен отчёт с графиками и анимацией.



СПИСОК ЛИТЕРАТУРЫ

1. Knuth D. E. Literate Programming // The Computer Journal. — 1984. — Feb. — Vol. 27, no. 2. — P. 97–111. — ISSN 1460-2067. — DOI: 10.1093/comjnl/27.2.97.
2. A Multi-Language Computing Environment for Literate Programming and Reproducible Research / E. Schulte [et al.] // Journal of Statistical Software. — 2012. — Vol. 46, no. 3. — ISSN 1548-7660. — DOI: 10.18637/jss.v046.i03.
3. The Story in the Notebook / M. B. Kery [et al.] // Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems. — ACM, 04/2018. — P. 1– 11. — DOI: 10.1145/3173574.3173748.

Speaker notes

